

# Dokumentasi Project CMS Wisata Puncak Mas



Dibuat Oleh :

Nama : Jon Renaldo Marbu,BN

NPM : 23312135

Kelas : IF 23 E

## Daftar ISI

|    |                                                            |    |
|----|------------------------------------------------------------|----|
| 1. | Landing CMS ( App/page.tsx ) .....                         | 4  |
| 2. | Login ( app/login/page.tsx ) .....                         | 5  |
| 3. | Dashboard (app/dashboard/page.tsx) .....                   | 8  |
| 4. | Detail Wisata ( app/detail_wisata/page.tsx ) .....         | 11 |
| 5. | Halaman Manajemen Transaksi (app/transaksi/page.tsx) ..... | 15 |
| 6. | Halaman Data Pengunjung app/pengunjung/page.tsx .....      | 20 |
| 7. | Layout Utama Aplikasi (app/layout.tsx) .....               | 26 |

## Daftar Gambar

|                                                              |    |
|--------------------------------------------------------------|----|
| Gambar 1 Sedikit Tampilan kode dari Landing CMS.....         | 4  |
| Gambar 2 Sedikit Tampilan kode dari app/login/page.tsx ..... | 5  |
| Gambar 3 Tampilan Halaman Login.....                         | 7  |
| Gambar 4 Tampilan Setelah Login dan dikirim Kode JWT.....    | 8  |
| Gambar 5 Sedikit Tampilan kode dari dashboard .....          | 8  |
| Gambar 6 Tampilan Halaman Dashboard .....                    | 11 |
| Gambar 7 Sedikit Tampilan kode dari wisata .....             | 11 |
| Gambar 8 Tampilan Halaman Wisata .....                       | 15 |
| Gambar 9 Sedikit Tampilan kode dari Transaksi .....          | 15 |
| Gambar 10 Tampilan Halaman Transaksi .....                   | 20 |
| Gambar 11 Sedikit Tampilan kode dari Pengunjung.....         | 20 |
| Gambar 12 Tampilan Halaman Pengunjung .....                  | 26 |
| Gambar 13 Tampilan print-ready layout.....                   | 26 |
| Gambar 14 Sedikit Tampilan Kode Layout .....                 | 26 |
| Gambar 15 Tampilan Layout .....                              | 27 |

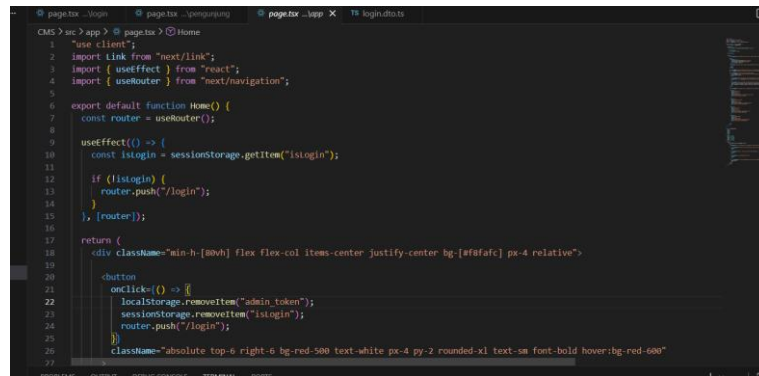
Framework yang digunakan pada project ini adalah:

- Next.js → Frontend
- NestJS → Backend API
- Tailwind CSS → Styling UI
- TypeScript → Bahasa Pemrograman

Project ini merupakan sistem CMS (Content Management System) untuk pengelolaan wisata Puncak Mas yang memiliki fitur:

- Login Admin
- Dashboard Statistik
- Detail Wisata
- Manajemen Transaksi
- Data Pengunjung
- Navigasi Admin Panel

## 1. Landing CMS ( App/page.tsx )



Gambar 1 Sedikit Tampilan kode dari Landing CMS

Halaman Landing CMS merupakan halaman utama atau beranda pusat kendali setelah admin berhasil masuk ke sistem. Halaman ini berfungsi sebagai gerbang utama navigasi yang menghubungkan pengguna ke seluruh modul pengelolaan, seperti Dashboard, Data Wisata, Transaksi, dan Data Pengunjung, serta dilengkapi dengan fitur proteksi keamanan autentikasi.

"use client"; import Link from "next/link"; import { useEffect } from "react"; import { useRouter } from "next/navigation";

- Penjelasan kode Direktif dan Import: Pernyataan "use client" memastikan komponen ini dirender di sisi browser pengguna karena memanfaatkan modul interaktif. Pustaka Link digunakan untuk navigasi antar halaman tanpa memuat ulang browser, useEffect menangani pengecekan kondisi saat halaman dimuat, dan useRouter mengontrol perpindahan rute halaman secara terprogram.

const router = useRouter(); useEffect() => { const isLogin = sessionStorage.getItem("isLogin"); if (!isLogin) { router.push("/login"); } }, [router];

- Penjelasan kode Proteksi Autentikasi Halaman: Fungsi di dalam hook useEffect akan memeriksa status login admin di dalam sessionStorage ketika halaman pertama kali

diakses. Jika kunci bernama "isLogin" tidak ditemukan, sistem menyimpulkan bahwa admin belum melakukan login yang sah, kemudian secara otomatis memindahkan paksa rute pengguna kembali ke halaman "/login" untuk mencegah akses ilegal.

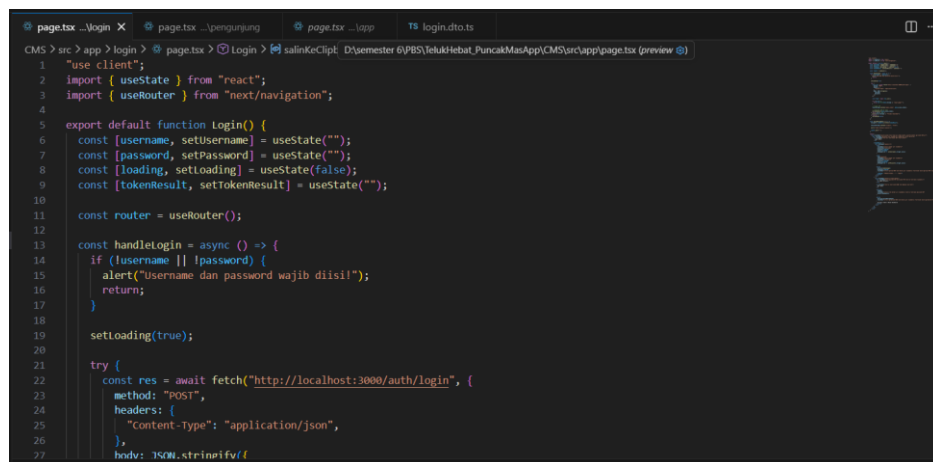
```
onClick={() => { localStorage.removeItem("admin_token");  
sessionStorage.removeItem("isLogin"); router.push("/login"); }}
```

- Penjelasan kode Fitur Keluar Sistem (Logout): Tombol ini menangani pembersihan data sesi pengguna. Saat diklik, aplikasi akan menghapus data "admin\_token" dari localStorage serta status "isLogin" dari sessionStorage secara permanen. Setelah seluruh data sesi dibersihkan, pengguna langsung diarahkan kembali menuju halaman masuk utama.
- Penjelasan kode Struktur Informasi Header: Bagian ini menyusun elemen judul utama dan sub-judul informasi halaman. Menggunakan penataan layout rata tengah dengan tipografi berbobot tebal untuk menegaskan identitas aplikasi sebagai pusat kendali admin pengelolaan destinasi wisata Puncak Mas Lampung.
- Penjelasan kode Kisi Navigasi Utama (Grid Menu): Menyusun empat buah kartu navigasi utama dengan memanfaatkan sistem tata letak grid yang dinamis dan responsif dari Tailwind CSS. Susunan menu otomatis menyesuaikan diri dari satu kolom pada perangkat ponsel pintar menjadi empat kolom sejajar ketika diakses menggunakan layar monitor komputer desktop.

```
function MenuCard({ href, emoji, title, desc, color }) { ... }
```

- Penjelasan kode Sub-Komponen MenuCard: Merupakan komponen kustom yang dapat digunakan kembali secara berulang untuk menampilkan pilihan menu navigasi. Komponen ini menerima properti berupa alamat tujuan tautan, ikon emoji, judul menu, deskripsi singkat, serta warna garis tepi dinamis yang akan aktif berubah warna saat kursor mouse diarahkan oleh pengguna.

## 2. Login ( app/login/page.tsx )



```
1 "use client";  
2 import { useState } from "react";  
3 import { useRouter } from "next/navigation";  
4  
5 export default function Login() {  
6   const [username, setUsername] = useState("");  
7   const [password, setPassword] = useState("");  
8   const [loading, setLoading] = useState(false);  
9   const [tokenResult, setTokenResult] = useState("");  
10  
11   const router = useRouter();  
12  
13   const handleLogin = async () => {  
14     if (!username || !password) {  
15       alert("Username dan password wajib diisi!");  
16       return;  
17     }  
18  
19     setLoading(true);  
20  
21     try {  
22       const res = await fetch("http://localhost:3000/auth/login", {  
23         method: "POST",  
24         headers: {  
25           "Content-Type": "application/json",  
26         },  
27         body: JSON.stringify({  
28           username, password,  
29         })  
30       });  
31       const data = await res.json();  
32       if (res.ok) {  
33         setTokenResult(data.token);  
34         router.push("/");  
35       } else {  
36         setTokenResult("");  
37       }  
38     } catch (error) {  
39       console.error("Login failed:", error);  
40     }  
41     setLoading(false);  
42   }  
43  
44   return (  
45     <div>  
46       <h1>Login</h1>  
47       <input type="text" value={username} onChange={e => setUsername(e.target.value)} />  
48       <input type="password" value={password} onChange={e => setPassword(e.target.value)} />  
49       <button onClick={handleLogin}>Login</button>  
50     </div>  
45   );  
46 }  
47
```

Gambar 2 Sedikit Tampilan kode dari app/login/page.tsx

Halaman Login digunakan sebagai pintu gerbang pengamanan autentikasi untuk memverifikasi identitas pengguna sebelum diberikan hak akses sebagai administrator. Halaman ini bertugas untuk menangani pengiriman kredensial berupa nama pengguna dan kata sandi menuju server otentikasi backend, menampung kode akses otorisasi yang dikembalikan, serta mengatur izin masuk ke dalam sistem pusat kendali.

```
"use client"; import { useState } from "react"; import { useRouter } from "next/navigation";
```

- Penjelasan kode Direktif dan Pustaka Impor: Deklarasi "use client" menandakan bahwa seluruh antarmuka komponen ini dijalankan di sisi browser karena melibatkan interaksi pengisian data oleh pengguna secara intensif. Pustaka useState diimpor untuk melacak setiap perubahan teks yang diketik serta status operasional program, sementara useRouter digunakan untuk melakukan pengalihan rute halaman web setelah verifikasi selesai dilakukan.

```
const [username, setUsername] = useState(""); const [password, setPassword] = useState(""); const [loading, setLoading] = useState(false); const [tokenResult, setTokenResult] = useState("");
```

- Penjelasan kode State Manajemen Autentikasi: Komponen ini mengelola empat bagian data status lokal. Variabel username dan password bertugas mengikat data teks input yang diisi oleh pengguna. Variabel loading berfungsi sebagai penanda biner untuk mematikan interaksi tombol saat proses pengecekan ke server sedang berjalan, sedangkan tokenResult bertugas menyimpan kode akses token keamanan (JSON Web Token) yang diterbitkan oleh server backend setelah data terbukti cocok.

```
if (!username || !password) { alert("Username dan password wajib diisi!"); return; }
```

- Penjelasan kode Validasi Input Form: Sebelum data dikirimkan melewati jaringan internet, fungsi ini melakukan pengecekan awal di sisi client. Jika salah satu atau kedua kolom pengisian ditemukan berada dalam kondisi kosong, aplikasi akan langsung membatalkan proses pengiriman dan menampilkan pesan peringatan singkat kepada pengguna untuk melengkapi kolom masukan data.

```
const res = await fetch("http://localhost:3000/auth/login", { method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify({ username, password }) }, {});
```

- Penjelasan kode Integrasi API Metode POST: Aplikasi melakukan komunikasi asinkron menuju endpoint API autentikasi lokal menggunakan metode HTTP POST. Data teks nama pengguna dan kata sandi yang berada di dalam state dibungkus dan dikonversi menjadi format teks JSON agar dapat diterima secara valid oleh arsitektur server backend NestJS.

```
if (!res.ok) { throw new Error(data.message || "Login gagal!"); } localStorage.setItem("admin_token", data.access_token); setTokenResult(data.access_token);
```

- Penjelasan kode Penanganan Respons dan Penyimpanan Token: Jika status respons yang dikembalikan oleh server bernilai salah, sistem akan melempar pesan kegagalan

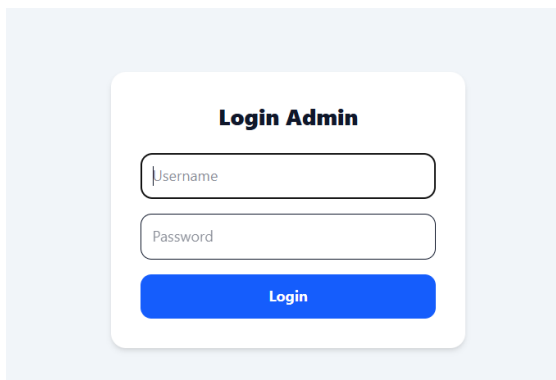
untuk memicu penanganan eror. Namun, jika respons berstatus sukses, kode token enkripsi yang berada di dalam properti `access_token` akan disimpan secara permanen ke dalam ruang penyimpanan lokal browser (`localStorage`) dengan nama kunci `"admin_token"`, sekaligus memperbarui isi status `tokenResult` untuk memicu perubahan tampilan antarmuka.

```
const salinKeClipboard = () => { navigator.clipboard.writeText(tokenResult);  
  sessionStorage.setItem("isLogin", "true"); alert("Token berhasil disalin!");  
  router.push("/"); };
```

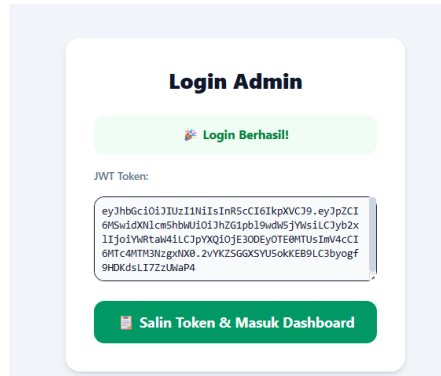
- Penjelasan kode Fitur Penyalinan Token dan Sesi Masuk: Fungsi ini dijalankan ketika pengguna menekan tombol persetujuan akhir setelah login berhasil. Sistem akan menyalin baris teks token yang tersimpan menuju papan klip (`clipboard`) komputer pengguna. Secara bersamaan, sistem menuliskan status `"isLogin"` bernilai `"true"` ke dalam `sessionStorage` sebagai penanda valid untuk melewati dinding pengaman halaman utama, lalu memindahkan rute halaman menuju beranda pusat kendali.

```
{!tokenResult ? ( ... ) : ( ... )}
```

- Penjelasan kode Render Antarmuka Kondisional: Komponen menerapkan sistem percabangan tampilan visual berdasarkan keberadaan nilai di dalam variabel `tokenResult`. Jika token masih berada dalam kondisi kosong, antarmuka akan menampilkan formulir masukan teks biasa yang berisi kolom nama pengguna, kata sandi, dan tombol verifikasi. Sebaliknya, ketika token sudah terisi, formulir otomatis disembunyikan dan diganti dengan panel pemberitahuan sukses yang menampilkan kotak teks kode token beserta tombol konfirmasi masuk dashboard.



*Gambar 3 Tampilan Halaman Login*



Gambar 4 Tampilan Setelah Login dan dikirim Kode JWT

### 3. Dashboard (app/dashboard/page.tsx)

```

1  "use client";
2  import { useEffect, useState } from "react";
3
4  type TransaksiType = {
5    id: number;
6    total: number;
7    jumlah: number;
8  };
9
10 export default function Dashboard() {
11   const [data, setData] = useState<TransaksiType>({});
12   const [loading, setloading] = useState(true);
13
14   const API_URL = "http://localhost:3000/wisata";
15
16   useEffect(() => {
17     const fetchData = async () => {
18       try {
19         const res = await fetch(API_URL, { cache: "no-store" });
20         const result = await res.json();
21         setData(result);
22       } catch (error) {
23         console.error("Gagal ambil data dashboard:", error);
24       } finally {
25         setloading(false);
26       }
27     };
28   });

```

Gambar 5 Sedikit Tampilan kode dari dashboard

Halaman Dashboard digunakan untuk menampilkan ringkasan data operasional sistem secara *real-time*. Halaman ini berfungsi sebagai panel pemantau bagi admin dengan menyajikan metrik penting seperti total transaksi, akumulasi pemasukan finansial, jumlah total pengunjung yang terdata, hingga indikator kesehatan sistem operasional backend.

"use client";

import { useEffect, useState } from "react";

"use client" Menandakan bahwa seluruh baris kode di dalam halaman ini akan dieksekusi di sisi *client* (browser). Hal ini wajib dideklarasikan karena komponen menggunakan fitur React Hooks seperti useState untuk manajemen data dan useEffect untuk proses pengambilan data eksternal.

```

type TransaksiType = {
  id: number;
  total: number;
  jumlah: number;

```



```
};
```

Definisi Tipe Data Struktur objek `TransaksiType` dibuat menggunakan TypeScript untuk memastikan data transaksi yang diterima dari API memiliki properti yang valid, yaitu `id` sebagai pengenal unik, `total` untuk nominal uang, dan `jumlah` untuk kuantitas tiket.

```
const [data, setData] = useState<TransaksiType[]>([]);
const [loading, setLoading] = useState(true);
```

State Lokal Komponen

`data`: Digunakan untuk menyimpan larik (*array*) objek transaksi hasil unduhan dari backend. Diinisialisasi dengan *array* kosong agar fungsi komputasi tidak mengalami *error* di awal render.

`loading`: Berfungsi sebagai indikator untuk mengetahui apakah proses penarikan data dari jaringan masih berjalan atau sudah selesai.

```
const API_URL = "http://localhost:3000/wisata";
```

Alamat API URL Variabel konstan yang menyimpan alamat *endpoint* REST API backend berbasis NestJS untuk mengambil sumber data mentah seputar transaksi wisata Puncak Mas.

```
useEffect(() => {
  const fetchData = async () => {
    try {
      const res = await fetch(API_URL, { cache: "no-store" });
      const result = await res.json();
      setData(result);
    } catch (error) {
      console.error("Gagal ambil data dashboard:", error);
    } finally {
      setLoading(false);
    }
  };
  fetchData();
}, []);
```

Penggunaan `useEffect` dan Fetch API Hook `useEffect` dengan array ketergantungan kosong memastikan fungsi `fetchData` langsung berjalan satu kali saat halaman pertama kali dibuka. Opsi `{ cache: "no-store" }` diterapkan agar browser selalu meminta data paling segar langsung dari server tanpa menyimpan data lama (*cache*). Blok *try-catch-finally* bertugas mengamankan jalannya program; jika terjadi kegagalan jaringan, eror akan ditangkap tanpa merusak UI, dan status `loading` otomatis dimatikan di akhir proses.

```
const totalTransaksi = data.length;
```

Total Transaksi Menghitung jumlah seluruh transaksi yang terjadi di Puncak Mas dengan cara membaca panjang atau total elemen yang tersimpan di dalam state `data`.

```
const totalPemasukan = data.reduce((sum, item) => sum + (item.total || 0), 0);
```

Total Pemasukan Menjumlahkan seluruh nominal pendapatan dengan mengiterasi isi data menggunakan fungsi `.reduce()`. Penggunaan logika `(item.total || 0)` berfungsi sebagai pengaman agar sistem tidak menghasilkan nilai kosong (*NaN*) jika terdapat baris data yang cacat atau bernilai *null*.

```
const totalPengunjung = data.reduce((sum, item) => sum + (item.jumlah || 0), 0);
```

Total Pengunjung Menerapkan logika fungsi `.reduce()` yang serupa dengan perhitungan pemasukan, namun kalkulasi kali ini difokuskan untuk mengumpulkan total kuantitas dari properti jumlah guna mengetahui angka pasti seluruh pengunjung yang datang.

```
const formatRupiah = (angka: number) => {  
  return new Intl.NumberFormat("id-ID", {  
    style: "currency",  
    currency: "IDR",  
    maximumFractionDigits: 0,  
  }).format(angka);  
};
```

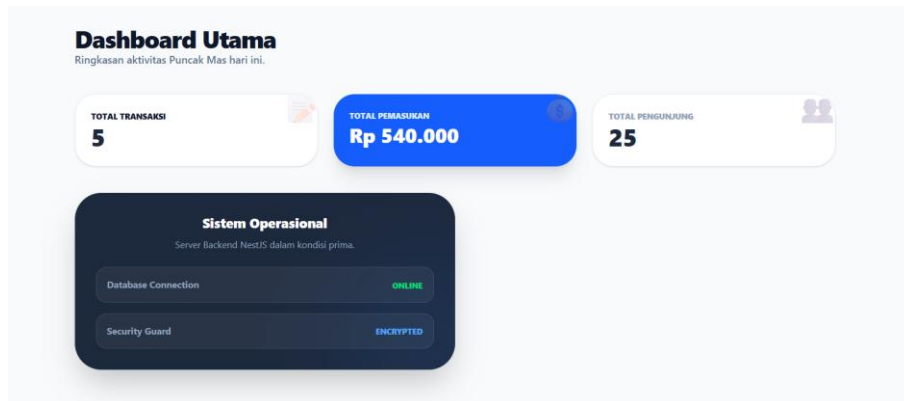
Fungsi Format Rupiah Mengubah tipe data angka mentah menjadi format teks mata uang Rupiah standar Indonesia. Pengaturan `maximumFractionDigits: 0` digunakan untuk menghilangkan dua angka nol desimal di belakang koma agar tampilan nominal pada kartu informasi terlihat lebih rapi.

```
    { /* STATS GRID */  
    <div className="grid grid-cols-1 md:grid-cols-3 gap-6 mb-10">  
      { /* Total Transaksi */  
      { /* Total Pemasukan */  
      { /* Total Pengunjung */  
    </div>
```

Desain Kartu Statistik Tampilan metrik menggunakan sistem Grid Layout Tailwind CSS yang responsif (1 kolom pada ponsel, otomatis menjadi 3 kolom pada layar komputer). Kartu "Total Pemasukan" diberikan warna biru kontras (`bg-blue-600`) dengan bayangan tebal untuk menerapkan teknik *Visual Hierarchy*, sehingga perhatian utama pengguna langsung tertuju pada informasi keuangan. Setiap kartu juga dilengkapi ornamen emoji dengan transparansi rendah (`opacity-20`) untuk mempercantik estetika visual.

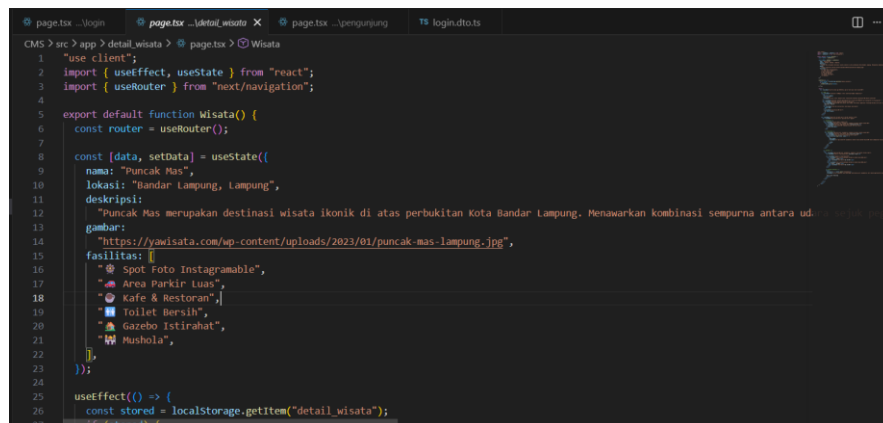
```
    { /* SISTEM OPERASIONAL */  
    <div className="w-full md:w-2/3 lg:w-1/2 bg-slate-800 p-8 rounded-[2.5rem] ..." >  
      <span>Database Connection</span> <span className="text-green-400">ONLINE</span>  
      <span>Security Guard</span> <span className="text-blue-400">ENCRYPTED</span>  
    </div>
```

Indikator Sistem Operasional Bagian bawah halaman diisi oleh panel khusus bertema gelap (bg-slate-800) yang berfungsi sebagai indikator visual untuk memantau kondisi backend aplikasi. Panel ini menampilkan status koneksi database (ONLINE) dan enkripsi keamanan (ENCRYPTED) guna memastikan seluruh sistem berjalan dalam kondisi prima.



Gambar 6 Tampilan Halaman Dashboard

#### 4. Detail Wisata ( app/detail\_wisata/page.tsx )



Gambar 7 Sedikit Tampilan kode dari wisata

- **Tujuan Halaman**

Tujuan dari halaman Detail Wisata adalah:

- Menampilkan informasi lengkap destinasi wisata.
- Memberikan gambaran visual melalui foto wisata.
- Menampilkan fasilitas yang tersedia.
- Menyediakan informasi harga tiket masuk.
- Mempermudah pengunjung melakukan pemesanan tiket.

- **Struktur Data Wisata**

Data wisata disimpan menggunakan React State dengan struktur sebagai berikut:

```
const [data, setData] = useState({
  nama: "Puncak Mas",
  lokasi: "Bandar Lampung, Lampung",
  deskripsi: "...",
  gambar: "...",
  fasilitas: [...]
});
```

## Penjelasan

| Atribut   | Fungsi                            |
|-----------|-----------------------------------|
| nama      | Menyimpan nama wisata             |
| lokasi    | Menyimpan lokasi wisata           |
| deskripsi | Menyimpan deskripsi wisata        |
| gambar    | Menyimpan URL gambar wisata       |
| fasilitas | Menyimpan daftar fasilitas wisata |

- **Penggunaan useState**

Pada halaman ini digunakan hook useState untuk menyimpan data wisata yang akan ditampilkan pada halaman.

```
const [data, setData] = useState(...)
```

### Fungsi

- Menyimpan data wisata secara dinamis.
- Memungkinkan data berubah tanpa perlu me-refresh halaman.
- Menjadi sumber data utama yang ditampilkan pada tampilan website.

- **Penggunaan useEffect**

```
useEffect(() => {
  const stored = localStorage.getItem("detail_wisata");
  if (stored) {
    setData(JSON.parse(stored));
  }
}, []);
```

### Fungsi

Kode ini digunakan untuk mengambil data wisata yang tersimpan pada browser menggunakan Local Storage.

### Alur Kerja

1. Halaman dibuka.
2. Sistem membaca data dengan key detail\_wisata.
3. Jika data ditemukan:
  - Data diubah dari format JSON menjadi object.
  - Data dimasukkan ke dalam state menggunakan setData().
4. Informasi wisata akan diperbarui secara otomatis.

### Manfaat

- Data tetap tersimpan meskipun halaman direfresh.
- Memungkinkan perubahan informasi wisata tanpa mengubah kode secara langsung.

- **Hero Section**

Hero Section merupakan bagian utama yang pertama kali dilihat oleh pengguna.

```
<img  
src={data.gambar}  
alt="wisata"  
>
```

**Fungsi**

Menampilkan gambar utama destinasi wisata sebagai banner halaman.

**Informasi yang Ditampilkan**

- Foto wisata
- Nama wisata
- Lokasi wisata
- Label "Destinasi Populer"

**Kelebihan**

- Memberikan tampilan visual yang menarik.
- Menampilkan identitas wisata secara jelas.
- Meningkatkan pengalaman pengguna.

- **Informasi Lokasi Wisata**

Lokasi wisata ditampilkan menggunakan kode berikut:

```
<p>  
{data.lokasi}  
</p>
```

**Fungsi**

Menampilkan lokasi destinasi wisata sehingga pengunjung mengetahui letak objek wisata yang akan dikunjungi.

Contoh:

Bandar Lampung, Lampung

- **Deskripsi Wisata**

Bagian deskripsi digunakan untuk menjelaskan informasi mengenai objek wisata.

```
<p>  
{data.deskripsi}  
</p>
```

**Fungsi**

Menampilkan informasi mengenai:

- Keunikan wisata
- Kondisi lingkungan
- Daya tarik wisata
- Aktivitas yang dapat dilakukan pengunjung

**Tujuan**

Memberikan gambaran kepada pengunjung sebelum memutuskan untuk berkunjung.

- **Fasilitas Wisata**

Daftar fasilitas ditampilkan menggunakan fungsi map().

```
data.fasilitas.map((item, index) => (
    <div key={index}>
        {item}
    </div>
))
```

### Fungsi

Menampilkan seluruh fasilitas yang tersedia pada wisata secara otomatis.

### Fasilitas yang Ditampilkan

- Spot Foto Instagramable
- Area Parkir Luas
- Kafe & Restoran
- Toilet Bersih
- Gazebo Istirahat
- Mushola

### Keuntungan Menggunakan map()

- Kode lebih singkat.
- Mudah menambah fasilitas baru.
- Data dapat ditampilkan secara otomatis.

- **Informasi Tiket**

Bagian sidebar digunakan untuk menampilkan informasi harga tiket masuk wisata.

### Harga Tiket

| Hari           | Harga    |
|----------------|----------|
| Senin – Jumat  | Rp20.000 |
| Sabtu – Minggu | Rp25.000 |

### Fungsi

Memberikan informasi biaya kunjungan kepada pengunjung sebelum melakukan pembelian tiket.

- **Tombol Pembelian Tiket**

Navigasi menuju halaman transaksi dilakukan menggunakan:

```
onClick={() => router.push("/transaksi")}
```

### Fungsi

Mengarahkan pengguna menuju halaman transaksi ketika tombol "**Beli Tiket Sekarang**" ditekan.

### Alur Kerja

1. Pengguna membuka Detail Wisata.
2. Membaca informasi wisata.
3. Menekan tombol "Beli Tiket Sekarang".
4. Sistem mengarahkan pengguna ke halaman Transaksi.
5. Pengguna dapat melakukan pembelian tiket.

## 4.12 Komponen yang Digunakan

| Komponen | Fungsi                |
|----------|-----------------------|
| useState | Menyimpan data wisata |

| Komponen  | Fungsi                            |
|-----------|-----------------------------------|
| useEffect | Mengambil data dari Local Storage |
| useRouter | Navigasi halaman                  |
| img       | Menampilkan gambar wisata         |
| map()     | Menampilkan daftar fasilitas      |
| Button    | Navigasi menu transaksi           |

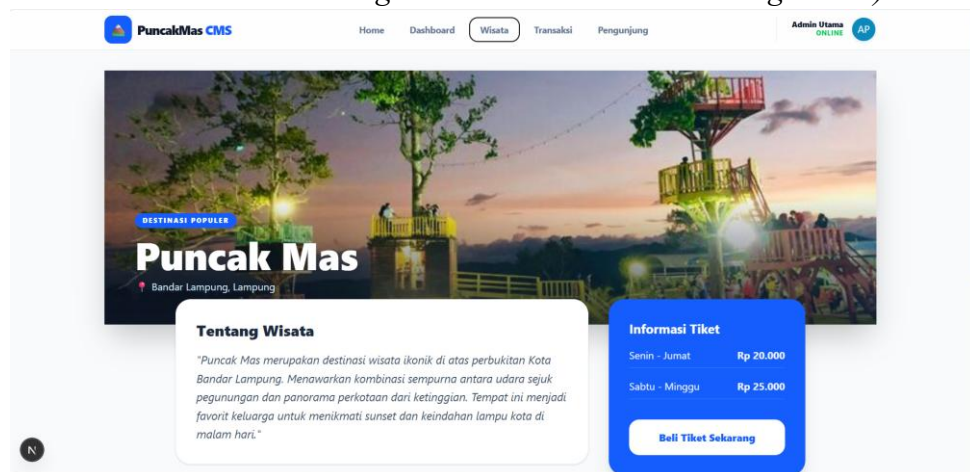
### • Tampilan Halaman

Pada halaman ini ditampilkan beberapa komponen utama:

1. Hero Banner Wisata
2. Nama dan Lokasi Wisata
3. Deskripsi Wisata
4. Daftar Fasilitas
5. Informasi Harga Tiket
6. Tombol Pembelian Tiket

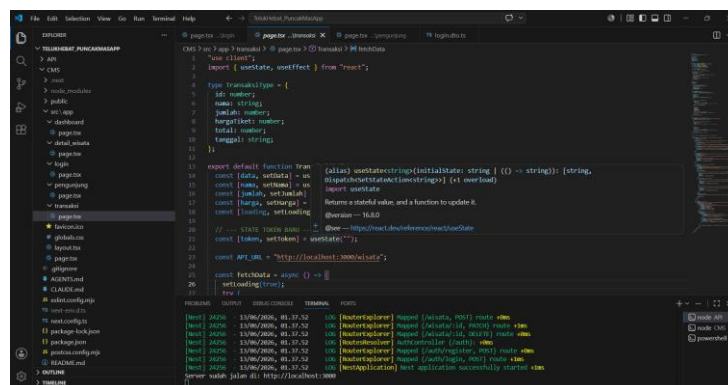
### Gambar 4.x Tampilan Halaman Detail Wisata

(Letakkan screenshot hasil running halaman Detail Wisata di bagian ini.)



Gambar 8 Tampilan Halaman Wisata

## 5. Halaman Manajemen Transaksi (app/transaksi/page.tsx)



Gambar 9 Sedikit Tampilan kode dari Transaksi

Halaman Manajemen Transaksi berfungsi sebagai pusat pencatatan, pemantauan, dan penghapusan data penjualan tiket wisata secara *real-time*. Halaman ini menerapkan sistem pengamanan ketat menggunakan otorisasi JSON Web Token (JWT) yang dilekatkan pada setiap permintaan mutasi data (tambah dan hapus) ke server backend NestJS, guna memastikan hanya administrator sah yang dapat memodifikasi data keuangan.

```
"use client";  
import { useState, useEffect } from "react";
```

Fungsi Direktif dan Impor Hooks Pernyataan "use client" wajib dideklarasikan di baris paling atas untuk mengaktifkan mode *Client Component*, sehingga Next.js mengizinkan eksekusi fitur interaktif di browser. Pustaka `useState` digunakan untuk mengelola status data dinamis di dalam memori peramban seperti input nama, jumlah tiket, harga, dan token, sedangkan `useEffect` difungsikan untuk memicu aksi penarikan data secara otomatis dari server backend sesaat setelah halaman selesai dimuat.

```
type TransaksiType = {  
  id: number;  
  nama: string;  
  jumlah: number;  
  hargaTiket: number;  
  total: number;  
  tanggal: string;  
};
```

Fungsi Definisi Tipe Data TypeScript Struktur objek `TransaksiType` dibuat menggunakan TypeScript untuk memastikan data transaksi yang mengalir di dalam aplikasi memiliki properti yang valid. Aturan ketat ini mengunci kolom `id` sebagai pengenalan unik, `nama` untuk identitas pengunjung, `jumlah` untuk kuantitas tiket, `hargaTiket` untuk nilai satuan, `total` untuk kalkulasi akhir, dan `tanggal` sebagai penanda waktu agar konsistensi data antara *frontend* dan *backend* tetap terjaga.

```
const [data, setData] = useState<TransaksiType[]>([]);  
const [nama, setNama] = useState("");  
const [jumlah, setJumlah] = useState(1);  
const [harga, setHarga] = useState(20000);  
const [loading, setLoading] = useState(false);  
const [token, setToken] = useState("");
```

Fungsi Deklarasi State Manajemen Komponen Aplikasi menginisialisasi enam buah variabel status lokal yang terikat dengan elemen antarmuka. Variabel `data` menampung larik (*array*) hasil unduhan dari backend, variabel `nama`, `jumlah`, dan `harga` mengikat setiap teks dan angka yang diketikkan pengguna pada formulir input secara dua arah (*two-way data binding*), variabel `loading` melacak status komunikasi jaringan untuk mematikan interaksi tombol saat proses berlangsung, dan variabel `token` digunakan sebagai wadah penampung kode JWT admin.

```
const fetchData = async () => {
```



```

        setLoading(true);
        try {
const res = await fetch(API_URL, { cache: "no-store" });
const result = await res.json();
setData(result);
        } catch (error) {
console.error("Gagal ambil data:", error);
        } finally {
setLoading(false);
        }
    };

```

Fungsi Penarikan Data dari Server (Read API) Fungsi `fetchData` melakukan permintaan HTTP GET asinkron menuju server backend. Opsi `{ cache: "no-store" }` diterapkan secara ketat untuk menonaktifkan sistem penyimpanan singgah (*caching*) browser, sehingga data tabel transaksi yang diterima dan ditampilkan selalu bersifat aktual, segar, dan *real-time* langsung mengambil dari database server NestJS.

```

const formatRupiah = (angka: number) => {
return new Intl.NumberFormat("id-ID", {
style: "currency",
currency: "IDR",
maximumFractionDigits: 0,
}).format(angka);
};

```

Fungsi Utilitas Format Mata Uang Rupiah Fungsi ini memanfaatkan objek standar JavaScript `Intl.NumberFormat` untuk mengubah format angka nominal mentah menjadi susunan teks mata uang Rupiah yang baku untuk wilayah Indonesia (id-ID). Pengaturan objek ini memaksa sistem menyertakan simbol mata uang "Rp" dan memotong angka desimal di belakang koma (`maximumFractionDigits: 0`) demi kerapian visual pada kartu laporan keuangan.

```

const handleSubmit = async () => {
if (!nama) return alert("Isi nama pengunjung!");
if (!token) return alert("Masukkan Kode Otorisasi Admin!");
const payload = {
Nama: nama,
Jumlah: jumlah,
Harga: harga,
Tanggal: new Date().toISOString().split('T')[0],
};

```

Fungsi Validasi Form dan Pembungkusan Data Payload Blok kode ini melakukan pemeriksaan awal di sisi klien untuk memastikan input nama dan token admin tidak kosong saat tombol simpan ditekan. Jika validasi lolos, seluruh isian formulir dikemas ke dalam objek payload berformat JSON, di mana kolom tanggal otomatis diisolasi dari waktu sistem saat ini menggunakan manipulasi string ISO untuk menghasilkan format tahun-bulan-tanggal (YYYY-MM-DD).

```

        try {
            const res = await fetch(API_URL, {
                method: "POST",
                headers: {
                    "Content-Type": "application/json",
                    "Authorization": `Bearer ${token}` // Kirim Token ke NestJS
                },
                body: JSON.stringify(payload),
            });

            if (res.ok) {
                fetchData();
                setNama("");
                setJumlah(1);
                setHarga(20000);
            } else {
                const errorRes = await res.json();
                alert(`Gagal: ${errorRes.message} || "Akses Ditolak!"`);
            }
        } catch (error) {
            alert("Koneksi gagal!");
        }
    });

```

**Fungsi Operasi Simpan Transaksi dengan Pengaman JWT (Create API) Potongan kode ini menangani pengiriman data menuju API backend menggunakan metode HTTP POST. Lapis keamanan utama diterapkan pada headers, di mana kode JWT admin disisipkan menggunakan skema otentikasi Bearer Token. Jika server backend memverifikasi token dan mengembalikan respons sukses (res.ok), aplikasi akan memicu fungsi fetchData untuk memperbarui tabel serta mereset seluruh kolom formulir ke kondisi semula.**

```

const handleDelete = async (id: number) => {
    if (!token) return alert("Masukkan Kode Otorisasi Admin untuk menghapus!");
    if (!confirm("Hapus transaksi ini?")) return;

    try {
        const res = await fetch(`${API_URL}/${id}`, {
            method: "DELETE",
            headers: {
                "Authorization": `Bearer ${token}` // Kirim Token ke NestJS
            }
        });

        if (res.ok) {
            fetchData();

```

```

    } else {
      const errorRes = await res.json();
      alert('Gagal Hapus: ${errorRes.message || "Akses Ditolak!"});
    }
  } catch (error) {
    console.error("Gagal hapus:", error);
  }
};

```

**Fungsi Operasi Hapus Data Berbasis Token (Delete API)** Fungsi `handleDelete` mendeteksi parameter ID transaksi spesifik yang akan dieliminasi melalui tombol aksi di baris tabel. Sistem memberikan lapis keamanan ganda berupa kewajiban pengisian token otorisasi admin pada komponen header HTTP metode DELETE serta menampilkan jendela dialog konfirmasi (*confirm dialog*) bawaan browser untuk mencegah ketidaksengajaan klik oleh operator.

TypeScript

```
const totalSemua = data.reduce((sum, item) => sum + item.total, 0);
```

**Fungsi Akumulasi Total Pendapatan (Derived State)** Variabel `totalSemua` dihitung dengan menerapkan metode bawaan JavaScript `.reduce()` yang berjalan mengiterasi atau memutar setiap elemen *array* di dalam state `data`. Fungsi ini bertugas menjumlahkan setiap nilai di dalam properti `item.total` secara akumulatif, menghemat memori RAM browser, dan hasilnya siap ditampilkan pada kartu informasi utama "Total Pemasukan".

```

<input
  type="password"
  placeholder="Masukkan kode untuk Simpan/Hapus..."
  className="..."
  value={token}
  onChange={(e) => setToken(e.target.value)}
/>

```

**Fungsi Input Kode Otorisasi Admin** Komponen masukan teks ini dikonfigurasi secara ketat menggunakan tipe `password` agar deretan karakter token JWT yang ditempelkan oleh operator tersamar dan tidak terlihat langsung di layar demi keamanan. Elemen ini terikat dua arah dengan state `token` melalui fungsi pengatur `onChange` untuk menyuplai data kunci keamanan utama pada fungsi `simpan` dan `hapus`.

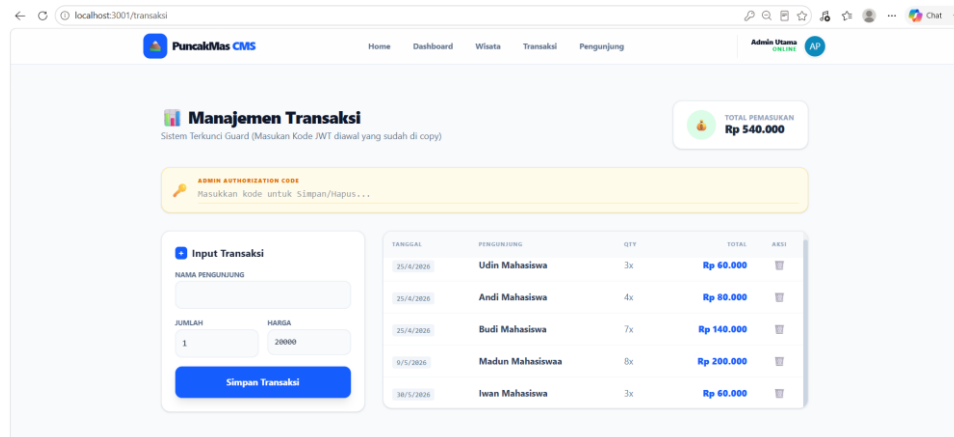
```

<div className="grid grid-cols-1 lg:grid-cols-12 gap-8">
  <div className="lg:col-span-4"> {/ * FORM INPUT */} </div>
  <div className="lg:col-span-8"> {/ * TABLE DATA */} </div>
</div>

```

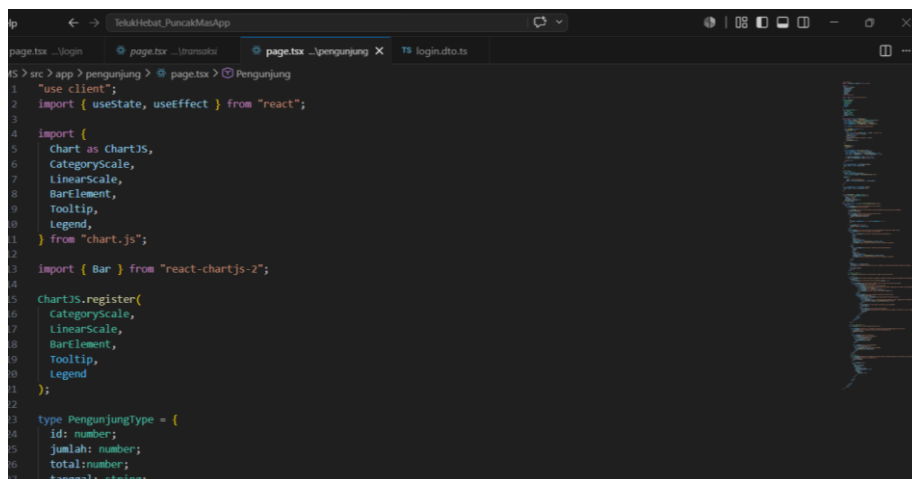
**Fungsi Kisi Layout Responsif Utama (Main Grid Layout)** Elemen pembungkus ini menerapkan sistem *Grid Layout* Tailwind CSS dengan pembagian total 12 kolom (`lg:grid-cols-12`). Susunan area kerja akan terbagi secara proporsional, di mana area formulir input transaksi mengambil porsi lebar 4 kolom (`lg:col-span-4`) dan komponen tabel riwayat data keuangan mendominasi porsi sisa sebanyak 8 kolom penuh (`lg:col-`

span-8), serta otomatis responsif bertumpuk vertikal jika dibuka lewat layar *handphone*.



Gambar 10 Tampilan Halaman Transaksi

## 6. Halaman Data Pengunjung app/pengunjung/page.tsx



Gambar 11 Sedikit Tampilan kode dari Pengunjung

Halaman Data Pengunjung dirancang sebagai sistem pelaporan eksekutif yang menyajikan data tren volume kunjungan wisatawan dan total omzet pendapatan secara *real-time*. Halaman ini dilengkapi dengan fitur filter rentang kalender, visualisasi diagram batang (*Chart.js*), serta penyesuaian tata letak khusus mode cetak (*print-ready layout*) untuk kebutuhan pembuatan dokumen fisik laporan manajemen.

```
"use client";
import { useState, useEffect } from "react";
```

```
import {
  Chart as ChartJS,
  CategoryScale,
  LinearScale,
  BarElement,
```

```

        Tooltip,
        Legend,
    } from "chart.js";

    import { Bar } from "react-chartjs-2";

    ChartJS.register(
        CategoryScale,
        LinearScale,
        BarElement,
        Tooltip,
        Legend
    );

```

**Fungsi Inisialisasi dan Registrasi Pustaka Visualisasi Grafik** Deklarasi "use client" memaksa Next.js mengeksekusi seluruh pustaka di sisi penjelajah web. Blok kode ini mengimpor komponen inti dari pustaka chart.js dan react-chartjs-2 untuk membangun diagram. Metode ChartJS.register() wajib dijalankan di awal untuk mendaftarkan skala kategori (CategoryScale), skala linier numerik (LinearScale), elemen geometri diagram batang (BarElement), serta fitur interaktif tooltip dan legenda agar grafik batang dapat dirender dengan sempurna.

```

type PengunjungType = {
    id: number;
    jumlah: number;
    total: number;
    tanggal: string;
};

```

**Fungsi Spesifikasi Struktur Data Pengunjung** Blok ini mendefinisikan tipe data TypeScript bernama PengunjungType untuk mengunci validitas struktur objek data dari server. Skema ini mewajibkan keberadaan kolom id sebagai referensi utama objek, jumlah untuk menampung angka total kuantitas wisatawan, total untuk menyimpan data akumulasi finansial rupiah, dan tanggal berbentuk string format waktu untuk keperluan pemetaan periodisasi laporan.

```

const [data, setData] = useState<PengunjungType[]>([]);
const [loading, setLoading] = useState(false);
const [tanggalAwal, setTanggalAwal] = useState("");
const [tanggalAkhir, setTanggalAkhir] = useState("");

```

**Fungsi Deklarasi State Laporan Interaktif** Aplikasi menginisialisasi empat variabel kondisi untuk mendukung sistem pencarian data dinamis. State data mengunci larik kosong terstruktur sesuai aturan PengunjungType[], variabel loading mengontrol tampilan indikator pemuatan data jaringan, sedangkan tanggalAwal dan tanggalAkhir difungsikan sebagai pengikat data (*data binding*) dari kolom input kalender filter pencarian.

```

const fetchData = async () => {
    setLoading(true);
    try {
        const res = await fetch(API_URL, { cache: "no-store" });
        const result = await res.json();
        setData(result);
    } catch (error) {
        console.error("Gagal load data:", error);
    } finally {

```

```

    setLoading(false);
  }
};

```

```

useEffect(() => {
  fetchData();
}, []);

```

### **Fungsi Sinkronisasi Data Laporan Terintegrasi (Read API)** Fungsi `fetchData`

mengimplementasikan operasi penarikan data asinkron (*async/await*) menuju endpoint API backend NestJS. Penambahan instruksi konfigurasi `{ cache: "no-store" }` bertujuan melompati memori singgah browser, memaksa aplikasi selalu mengambil data transaksi pengunjung paling mutakhir di database, yang dipicu secara otomatis satu kali di awal siklus hidup halaman melalui hook `useEffect`.

```

    const filteredData = data.filter((item) => {
      const tanggalData = new Date(item.tanggal);
      const awal = tanggalAwal ? new Date(tanggalAwal) : null;
      const akhir = tanggalAkhir ? new Date(tanggalAkhir) : null;

      if (awal && tanggalData < awal) return false;
      if (akhir && tanggalData > akhir) return false;
      return true;
    });

```

**Fungsi Komputasi Penyaringan Berdasarkan Kalender** Fungsi ini menerapkan metode `.filter()` untuk menyaring koleksi data transaksi berdasarkan input tanggal yang dipilih operator. Sistem mengubah string tanggal dari database dan input formulir menjadi objek `Date` JavaScript yang sebanding. Data transaksi akan dieliminasi dari daftar jika posisinya berada sebelum rentang waktu awal atau setelah rentang waktu akhir, menghasilkan larik data baru bernama `filteredData`.

```

    const totalPengunjung = filteredData.reduce(
      (sum, item) => sum + item.jumlah,
      0
    );

```

**Fungsi Kalkulasi Agregat Volume Wisatawan** Variabel `totalPengunjung` menghitung jumlah total wisatawan secara otomatis menggunakan fungsi tingkat tinggi `.reduce()`. Operasi ini berjalan mengiterasi larik data yang telah lolos penyaringan filter kalender (`filteredData`), menjumlahkan nilai properti `item.jumlah` ke dalam akumulator `sum` yang dimulai dari nilai awal nol, guna disajikan pada komponen kartu ringkasan.

```

    const chartData = {
      labels: filteredData.map((item) =>
        new Date(item.tanggal).toLocaleDateString("id-ID")
      ),
      datasets: [
        {
          label: "Jumlah Pengunjung",
          data: filteredData.map((item) => item.jumlah),
        },
      ],
    };

```

**Fungsi Transformasi Struktur Data Diagram Batang** Kode ini bertugas memetakan susunan array `filteredData` menjadi skema objek parameter yang dikenali oleh komponen grafik *Chart.js*. Properti labels diisi oleh larik string tanggal yang telah diformat ke standar lokal Indonesia (id-ID) menggunakan metode `.map()`, sedangkan properti data di dalam array datasets mengisolasi kumpulan angka volume pengunjung untuk disajikan menjadi grafik balok vertikal.

```
const totalPemasukan = filteredData.reduce(
  (sum, item) => sum + (item.total || 0),
  0
);
```

**Fungsi Akumulasi Finansial Omzet Kunjungan** Sama halnya dengan perhitungan total kuantitas orang, variabel `totalPemasukan` melakukan akumulasi total nilai nominal uang dari data yang tersaring. Sintaks ini dilengkapi pengaman siber berupa logika fallback *Short-Circuit Evaluation* (`item.total || 0`) untuk mengantisipasi nilai kosong (*null/undefined*) pada database agar kalkulasi tidak menghasilkan nilai eror fatal *Not a Number* (NaN).

```
const formatRupiah = (angka: number) => {
  return new Intl.NumberFormat("id-ID", {
    style: "currency",
    currency: "IDR",
    maximumFractionDigits: 0,
  }).format(angka);
};
```

```
const tanggalCetak = new Date().toLocaleDateString("id-ID");
```

**Fungsi Konversi Lokalisasi Keuangan dan Tanggal Cetak** Fungsi `formatRupiah` menggunakan standardisasi `Intl.NumberFormat` untuk mengubah angka numerik mentah menjadi format string mata uang resmi rupiah dengan pemisah ribuan titik. Sementara itu, variabel `tanggalCetak` mengambil waktu hari ini dari sistem komputer klien dan mengubahnya ke format lokal untuk disematkan sebagai penanda validitas pada berkas cetak fisik.

```
<h1 className="text-3xl font-extrabold text-slate-900 mb-8 text-center print:hidden">
  🧑 Data Pengunjung
</h1>
<div className="hidden print:block text-center mb-8">
  <h2 className="text-2xl font-bold">LAPORAN PENGUNJUNG PUNCAK MAS</h2>
  <p>Periode: {tanggalAwal || "-"} s/d {tanggalAkhir || "-"}</p>
  <p className="text-sm mt-2">Tanggal Cetak: {tanggalCetak}</p>
</div>
```

**Fungsi Kondisional Kop Surat Mode Cetak Dokumen** Elemen ini menerapkan teknik manipulasi tampilan responsif cetak dari utility class Tailwind. Judul antarmuka `<h1>` diberikan kelas `print:hidden` sehingga otomatis menghilang saat dokumen dicetak. Sebaliknya, komponen kontainer `<div>` di bawahnya memiliki kelas `hidden print:block` yang membuatnya tidak terlihat di layar monitor tetapi otomatis muncul sebagai Kop Surat resmi berlambang judul laporan dan batasan periode ketika tombol cetak dieksekusi.

```
JavaScript
<div className="flex flex-col md:flex-row justify-between items-center gap-4 mb-6">
  <div className="flex flex-wrap gap-3 print:hidden">
    <input type="date" value={tanggalAwal} onChange={(e) => setTanggalAwal(e.target.value)} />
    <input type="date" value={tanggalAkhir} onChange={(e) => setTanggalAkhir(e.target.value)} />
  </div>
  <button onClick={() => window.print()} className="... print:hidden">
```

```

 Cetak Laporan
</button>
</div>

```

**Fungsi Panel Kontrol Navigasi dan Pemicu Printer** Bagian ini menyediakan dua buah elemen input tanggal bertipe date untuk memicu pembaruan state pencarian. Di sebelahnya terdapat komponen tombol (*button*) yang mengikat fungsi bawaan browser window.print() pada event klik. Seluruh elemen kontrol pencarian dan tombol cetak ini dibubuhi kelas print:hidden agar kertas hasil cetakan printer bersih dari komponen-komponen tombol interaktif.

```

<div className="md:col-span-2 bg-white rounded-2xl shadow-sm border border-slate-200 overflow-
  hidden h-[420px] flex flex-col print:h-auto">
  <div className="overflow-y-auto h-[420px] print:h-auto print:overflow-visible">
    <table className="w-full text-left">
      {/* Struktur Head & Body Table */}
    </table>
  </div>
</div>

```

**Fungsi Kontainer Tabel Adaptif Lintas Media** Komponen pembungkus tabel diatur menggunakan kelas h-[420px] overflow-y-auto untuk menciptakan area gulir (*scrollable area*) internal di layar monitor browser agar menghemat ruang. Namun, untuk mencegah data tabel terpotong saat dicetak ke kertas, diterapkan modifikasi kelas print:h-auto print:overflow-visible yang memaksa kontainer melar secara vertikal menampilkan seluruh baris data secara utuh.

```

<thead className="sticky top-0 z-10 bg-slate-50">
  <tr className="...">
    <th className="p-4">Tanggal</th>
    <th className="p-4 text-center">Jumlah</th>
    <th className="p-4 text-right print:hidden">Status</th>
  </tr>
</thead>

```

**Fungsi Header Tabel dengan Kriteria Cetak Selektif** Struktur bagian kepala tabel dikonfigurasi dengan properti sticky top-0 agar posisinya tetap membeku di bagian atas saat daftar baris di dalam browser digulir ke bawah. Kolom ketiga yang berisi tajuk "Status" dipasang utility class print:hidden, karena informasi status pencatatan internal sistem tersebut dinilai tidak perlu ikut dicetak ke dalam dokumen laporan fisik manajemen.

```

    {filteredData.map((item) => (
      <tr key={item.id} className="hover:bg-slate-50 transition-colors">
        <td className="p-4">
          <span className="text-xs font-mono bg-slate-100 px-2 py-1 rounded text-slate-500">
            {new Date(item.tanggal).toLocaleDateString("id-ID")}
          </span>
        </td>
        <td className="p-4 text-center font-bold text-slate-800">{item.jumlah}</td>
        <td className="p-4 text-right print:hidden">
          <span className="bg-green-100 text-green-600 ...">Recorded</span>
        </td>
      </tr>
    ))}

```

**Fungsi Render Iterasi Baris Data Dinamis** Fungsi utama baris tabel ini menggunakan metode ekspresi .map() untuk menduplikasi elemen baris <tr> berdasarkan volume data di dalam larik filteredData. Atribut key={item.id} wajib disisipkan sebagai ID pelacak performa React. Di dalam



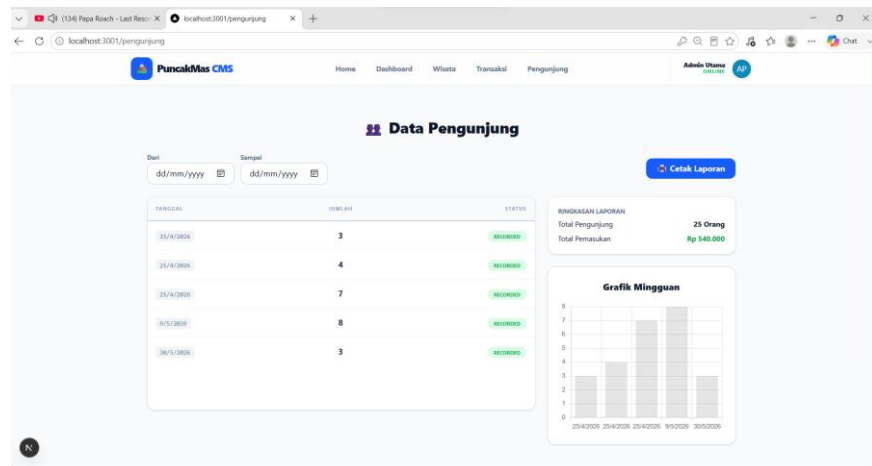
baris ini, data tanggal diubah ke format lokal Indonesia dan lencana (*badge*) status "Recorded" disembunyikan dari kertas cetak melalui instruksi `print:hidden`.

```
<div className="bg-white p-5 rounded-2xl border border-slate-200 shadow-sm print:mt-6">
  <p className="text-xs uppercase font-bold text-slate-500 mb-2">Ringkasan Laporan</p>
  <div className="space-y-2 text-sm">
    <div className="flex justify-between">
      <span>Total Pengunjung</span>
      <span className="font-bold">{totalPengunjung} Orang</span>
    </div>
    <div className="flex justify-between">
      <span>Total Pemasukan</span>
      <span className="font-bold text-green-600">{formatRupiah(totalPemasukan)}</span>
    </div>
  </div>
</div>
```

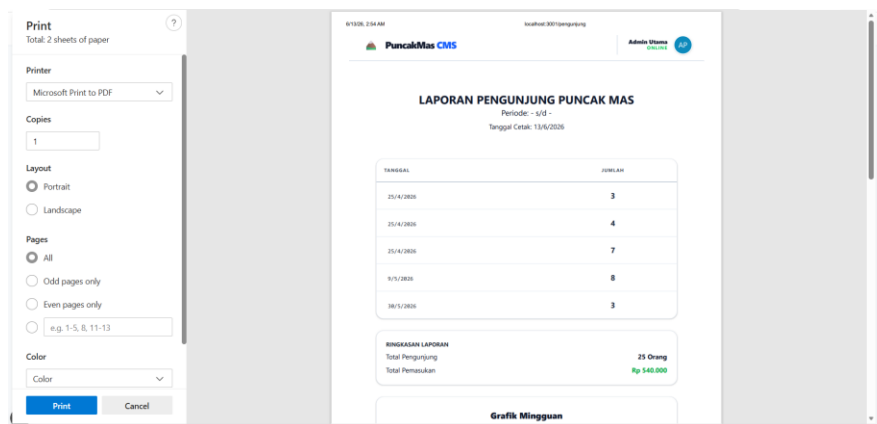
**Fungsi Komponen Panel Ringkasan Finansial** Komponen kotak ringkasan ini berfungsi menyajikan data kesimpulan analitik dari hasil kalkulasi variabel `totalPengunjung` dan fungsi `formatRupiah(totalPemasukan)`. Penambahan kelas CSS `print:mt-6` memberikan instruksi margin pembatas atas otomatis sebesar 24 piksel khusus pada dokumen cetak, memastikan posisi ringkasan terpisah secara rapi dari tabel di atasnya.

```
<div className="bg-white p-6 rounded-2xl border border-slate-200 shadow-sm text-center
  print:block">
  <h3 className="text-lg font-black mb-4">Grafik Mingguan</h3>
  <div className="h-64">
    <Bar
      data={chartData}
      options={{
        responsive: true,
        maintainAspectRatio: false,
        plugins: { legend: { display: false } },
      }}
    />
  </div>
</div>
```

**Fungsi Rendering Diagram Batang Interaktif** Komponen `<Bar />` bertindak sebagai perantara visual yang merender data objek `chartData` menjadi grafik analitik. Parameter `responsive: true` dan `maintainAspectRatio: false` memastikan grafik bersifat elastis mengikuti dimensi ruang kontainer tinggi 64 unit (`h-64`). Komponen ini dikunci dengan kelas `print:block` guna menjamin visualisasi grafik ikut terarsip secara rapi di dalam cetakan kertas laporan peninjauan.



Gambar 12 Tampilan Halaman Pengunjung



Gambar 13 Tampilan print-ready layout

## 7. Layout Utama Aplikasi (app/layout.tsx)

```
1  "use client";
2
3  import type { Metadata } from "next";
4  import { Geist, Geist_Mono } from "next/font/google";
5  import "./globals.css";
6  import Link from "next/link";
7  import { usePathname } from "next/navigation";
8
9  const geistSans = Geist({
10   variable: "--font-geist-sans",
11   subsets: ["latin"],
12 });
13
14  const geistMono = Geist_Mono({
15   variable: "--font-geist-mono",
16   subsets: ["latin"],
17 });
18
19  export default function RootLayout({
20   children,
21 }) {
22   children: React.ReactNode;
23 }
24
25 const pathname = usePathname();
26
27 // ...
```

Gambar 14 Sedikit Tampilan Kode Layout

### Fungsi Halaman

File ini merupakan layout utama yang digunakan di seluruh halaman aplikasi.

### Metadata

`export const metadata`

## Fungsi

Digunakan untuk:

- Judul website
- Deskripsi website

## Navbar

### Fungsi

Digunakan sebagai navigasi utama menuju:

- Home
- Dashboard
- Wisata
- Transaksi
- Pengunjung

## Komponen NavLink

### Fungsi

Membuat link navigasi lebih rapi dan reusable.

## Main Content

```
<main>
{children}
</main>
```

## Fungsi

Digunakan untuk menampilkan isi halaman sesuai route yang dibuka.

## Footer

### Fungsi

Menampilkan copyright aplikasi.



Gambar 15 Tampilan Layout